

M09 / PROTECTED ANSWERS

M09 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

A01 - DE09-01 | ETL vs ELT 3

A02 - DE09-02 | Layer contracts: raw, staging, intermediate, serving 3

A03 - DE09-03 | Deterministic transformation 4

A04 - DE09-04 | Deterministic deduplication and conflicts 4

A05 - DE09-05 | Schema drift and contract failure 5

A06 - DE09-06 | Reconciliation controls 5

A07 - DE09-07 | File hashes, watermarks and late change boundaries 6

A08 - DE09-08 | Retry, quality failure and post-staging failure 6

A09 - DE09-09 | Lineage and run metadata 6

A10 - DE09-10 | Reliable batch operating loop 7

A11 - DE09-11 | dbt source() vs ref() and DAG 7

A12 - DE09-12 | dbt tests and what PASS means 8

A13 - DE09-13 | Incremental boundary with >= max(updated_at) 8

A14 - DE09-14 | Fresh 14-day late-correction design 9

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a saved closed-book attempt. Each answer gives a concrete solution, a proof path, common mistakes, validation, and a changed retry. Do not overwrite your first attempt.

A01 - DE09-01 | ETL vs ELT

One acceptable solved answer

Example ETL: extract order files -> Python/Spark validates/transforms -> load curated warehouse table. Example ELT: land raw files/tables first -> SQL/dbt transforms inside the analytical platform. In both, preserve replayable source evidence and publish/state only after deterministic validation.

Worked proof / example

Draw source -> durable raw -> transform engine -> serving target and mark where the first durable load occurs.

Common beginner mistakes

Calling any Python script ETL and any SQL query ELT without locating the load boundary; cleaning before preserving raw evidence.

Validate your own attempt

Another engineer can reproduce what arrived, where compute happened, what failed safely and how to replay.

Changed retry

Redesign a 20 GB nightly orders flow once as ETL and once as ELT; name one tradeoff of each.

A02 - DE09-02 | Layer contracts: raw, staging, intermediate, serving

One acceptable solved answer

Order: raw -> staging -> intermediate -> serving. Raw preserves arrival; staging enforces/normalizes source contract; intermediate holds reusable business logic such as dedup/join; serving exposes stable consumer grain/semantics.

Worked proof / example

For an order pipeline, raw keeps original fields, staging parses timestamps/types, intermediate resolves deterministic latest-order candidates, serving publishes one order grain with net_sales.

Common beginner mistakes

Putting business aggregation in raw; using staging as a permanent serving table; skipping a layer just because its name seems optional without explaining the contract.

Validate your own attempt

For each layer state grain, owner, allowed transformations and replay responsibility.

Changed retry

Map a customer source through the four layers and identify where duplicate resolution belongs.

A03 - DE09-03 | Deterministic transformation

One acceptable solved answer

For qty=2, unit_price=850, discount=0.10: net_sales = 2 x 850 x (1 - 0.10) = 1530.00. Avoid arbitrary file/row order, wall-clock dependence, uncontrolled float/rounding, and unspecified tie winners.

Worked proof / example

```
from decimal import Decimal, ROUND_HALF_UP
q=Decimal('2'); p=Decimal('850'); d=Decimal('0.10')
net=(q*p*(Decimal('1')-d)).quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)
assert net==Decimal('1530.00')
```

Common beginner mistakes

Binary float money with unspecified rounding; using now() as a transformation input; last-row-wins without deterministic ordering.

Validate your own attempt

Re-run with input rows shuffled and prove the same output/key set/totals.

Changed retry

Use qty=3, price=199.95, discount=0.075 and state the rounding rule.

A04 - DE09-04 | Deterministic deduplication and conflicts

One acceptable solved answer

O6101: 09:00 qty2 is accepted; 08:00 qty1 is older and discarded. O6102: both 08:30 rows are conflicting maxima, so neither is arbitrarily accepted; quarantine the conflict for review.

Worked proof / example

```
# pseudocode
max_ts = group.updated_at.max()
candidates = group[group.updated_at==max_ts]
if len(candidates)==1: accept(candidates.iloc[0])
elif candidates.drop(columns=['ingest_meta']).drop_duplicates().shape[0]==1: accept(candidates.iloc[0])
else: quarantine_all(candidates, 'ambiguous_latest')
```

Common beginner mistakes

ROW_NUMBER() choosing one tied maximum without a source tie-break contract.

Validate your own attempt

Accepted + discarded + quarantined must reconcile to source rows and accepted business keys must be unique.

Changed retry

Add two identical max-timestamp duplicates and decide whether your declared policy accepts one or quarantines both.

A05 - DE09-05 | Schema drift and contract failure

One acceptable solved answer

Preserve/hash raw unchanged; record observed vs expected schema; do not publish incompatible serving data; do not advance successful state until a deliberate compatible mapping/contract version is approved.

Worked proof / example

If region becomes sales_region and discount_pct disappears, staging should fail the required-column/contract check before serving publish. The run manifest should still point to preserved raw evidence.

Common beginner mistakes

Silently filling missing discount with zero; renaming fields ad hoc without versioning; moving watermark despite failed publish.

Validate your own attempt

Target and committed state remain unchanged while raw/failed-run evidence exists.

Changed retry

Make one optional column appear and one required column disappear; classify them differently.

A06 - DE09-06 | Reconciliation controls

One acceptable solved answer

For the duplicate fixture: 4 source = 1 accepted/staged + 2 quarantined + 1 discarded. Independent controls include key-set membership/uniqueness, money totals, rejection-reason counts and state position.

Worked proof / example

```
assert source_rows == accepted_rows + quarantined_rows + discarded_rows
assert accepted.order_id.is_unique
assert accepted.net_sales.sum() == expected_money_total
```

Common beginner mistakes

Row count only; comparing totals after fan-out; not reconciling quarantine/discard populations.

Validate your own attempt

Use at least one population equation, one key control and one business-value control.

Changed retry

Change one amount and add one rejected row; predict all affected controls before running.

A07 - DE09-07 | File hashes, watermarks and late change boundaries

One acceptable solved answer

A file hash proves exact file identity; a watermark selects progress in an ordered changing source. Neither alone catches arbitrary late corrections: an update older than the watermark can be missed, and a new delivery can contain valid changes outside the simple boundary. Use CDC/change version, overlap/rebuild or reconciliation according to the source contract.

Worked proof / example

Example: committed watermark=2026-09-10. A correction to an order carries updated_at=2026-09-01. A predicate updated_at > watermark never sees it even though the source changed.

Common beginner mistakes

Calling a watermark “exactly once”; assuming filename/hash processing prevents row-level late updates.

Validate your own attempt

State the latest event/update class your method can still miss.

Changed retry

Design a 3-day lookback and state what happens to a correction 10 days late.

A08 - DE09-08 | Retry, quality failure and post-staging failure

One acceptable solved answer

Database lock: bounded retry; target/state advance only on eventual success. 80% invalid data: deterministic quality failure, no blind retry, target/state unchanged. Failure after staging: preserve staging/run evidence but do not publish target or advance committed state; replay safely later.

Worked proof / example

Use a run-status table: STARTED -> STAGED -> VALIDATED -> PUBLISHED -> COMMITTED. Only COMMITTED updates live progress.

Common beginner mistakes

Retrying bad data indefinitely; committing watermark because staging succeeded.

Validate your own attempt

For each failure class answer: is retry useful, what durable evidence remains, did target change, did state advance?

Changed retry

Classify a 429 API response, permission-denied target write and schema drift.

A09 - DE09-09 | Lineage and run metadata

One acceptable solved answer

Trace serving O6007 -> source_run_id -> successful batch manifest -> source filename/hash -> preserved raw O6007 -> transformation rule -> serving value. The lineage must be reproducible without relying on memory.

Worked proof / example

```
SELECT s.order_id,s.source_run_id,r.source_file,r.source_sha256
FROM serving_orders s
JOIN pipeline_runs r USING(source_run_id)
WHERE s.order_id='06007';
```

Common beginner mistakes

Lineage that stops at “came from staging”; storing run ID but not source identity/hash/rule version.

Validate your own attempt

Pick one serving row and recover source file/hash, run, rule/model version and key transformation.

Changed retry

Trace a quarantined row instead of an accepted serving row.

A10 - DE09-10 | Reliable batch operating loop

One acceptable solved answer

One acceptable ten-step loop: 1 receive/fingerprint; 2 preserve raw; 3 validate contract/types; 4 deterministic transform; 5 dedup/conflict handling; 6 reconcile; 7 quality gate; 8 transactional publish; 9 target controls; 10 atomically commit state/final manifest.

Worked proof / example

Failure before step 8 may keep raw/staging evidence but must not imply target success; failure at/after publish requires transaction/idempotency controls so rerun cannot double business data.

Common beginner mistakes

Moving checkpoint between transform and publish; omitting reconciliation; treating log “success” as target validation.

Validate your own attempt

For each step identify its input, durable output and failure-safe next state.

Changed retry

Reorder the steps for a database-source incremental pipeline and justify any change.

A11 - DE09-11 | dbt source() vs ref() and DAG

One acceptable solved answer

source() points to declared external/raw input; ref() points to another dbt model and creates a managed dependency. Supplied graph: raw.raw_orders -> stg_orders -> int_orders_deduped -> fct_orders; the incremental fact also consumes int_orders_deduped.

Worked proof / example

```
-- staging
select * from {{ source('raw','raw_orders') }}

-- mart
select * from {{ ref('int_orders_deduped') }}
```

Common beginner mistakes

Hard-coding model table names and losing DAG dependency; using ref() for an unmanaged source table.

Validate your own attempt

dbt parse/docs should show the intended edges once run on the learner PC.

Changed retry

Add a new dim_customer model and state which downstream refs should depend on it.

A12 - DE09-12 | dbt tests and what PASS means

One acceptable solved answer

Generic examples: not_null, unique, accepted_values, relationships. Singular example: SQL that returns rows where net_sales < 0. A passing data test means the failure query returned zero rows under its configured severity; it does not prove every possible property.

Worked proof / example

```
-- singular test
select * from {{ ref('fct_orders') }}
where net_sales < 0
```

Common beginner mistakes

Interpreting one passing test as full correctness; writing a test whose query returns valid rows rather than failing rows.

Validate your own attempt

For every test, state what bad rows it returns and what important risk it does not cover.

Changed retry

Write a singular test for duplicate business keys at the serving grain.

A13 - DE09-13 | Incremental boundary with >= max(updated_at)

One acceptable solved answer

Using updated_at >= max(target.updated_at) re-reads rows tied at the current boundary. With unique_key=order_id, those repeated keys can be safely upserted in the exercise. It still misses arbitrary corrections whose updated_at is older than the boundary.

Worked proof / example

```
{% if is_incremental() %}
where updated_at >= (select coalesce(max(updated_at),'1900-01-01') from {{ this }})
```

```
{% endif %}
```

Common beginner mistakes

Using `>` and skipping newly arrived boundary ties; claiming `>=` solves all late data.

Validate your own attempt

Add a new row at exactly the current max timestamp and prove it is not skipped; then explain why a one-month-old correction still can be.

Changed retry

Replace the boundary with a 1-day lookback and state added compute/upsert requirements.

A14 - DE09-14 | Fresh 14-day late-correction design

One acceptable solved answer

One safe design: each run selects source changes for the last 14 days (or uses a source change sequence/CDC if available), deterministically resolves versions/conflicts, upserts by business key, reconciles the rebuilt window, publishes transactionally, then advances committed state. Periodically run a wider reconciliation/backfill for changes beyond the declared window.

Worked proof / example

Cost: repeated scanning/compute and target upserts for 14 days; stronger tests/reconciliation take longer. Benefit: corrections within the stated SLA are recoverable and replay is idempotent.

If the source provides authoritative monotonic change sequence/CDC, prefer that over inferring change from mutable business timestamps.

Common beginner mistakes

14-day lookback without dedup/upsert; advancing watermark before target validation; claiming the window covers changes older than 14 days.

Validate your own attempt

Test identical replay, correction 7 days late, correction 20 days late and equal-version conflict; each must have a declared outcome.

Changed retry

Change the SLA to 48 hours and estimate how the scan/reconciliation cost changes.